catanrl v2

experiment log

living document · last updated 17 october 2026 · github.com/mcrco/catanrl-v2 · wandb: catanrl-v2

the goal of this repo is a recipe for the best catan bot out there, for any number of players on any map, with every experiment a config file under `configs/experiments/` and every number below traceable to an eval record under `runs/`. v1 (catanrl) got a two-player bot to about 90% against catanatron's `ValueFunctionPlayer` (F); v2 rewrote the stack around a C++ engine and pufferlib and is past 95% in 2p from scratch. this log is organized by question: what architecture, what the current best recipe is and what it measures, and the list of ideas i'm working through (each with a checkbox).

1 basic arch ablations

previously did some informal arch ablations, and realized that the graph network proposed in canopy is the fastest, most efficient, and most generalizable (can scale to arbitrary maps since you predict board actions per intersection/road/tile). but would still be interested in doing a formal ablation with MLP, LSTM, CNN (like in the old catanrl), and GNN (canopy) eventually.

what the informal ablations (v1, branch `arch-ablation`) actually showed, so i don't have to redo them:

- on a teacher-neutral target (predicting the terminal outcome of held-out games, win/draw/loss cross-entropy in nats, 0.693 is a coin flip) the ranking was graph < MLP < CNN: `graph-s` 0.396 (0.5M params), `mlp-512` 0.464 (2.4M), xdim CNN 0.83 (16.8M). the CNN predicted the winner worse than a coin flip and its value head memorized within a few hundred steps. a graph transformer over the same node and tile tokens (`gtx-s`, 0.397) tied message passing, so what matters is the structured head (per-node settlement/city logits, per-edge road logits, per-tile robber logits), not the exact mixing layer.
- imitation loss is biased toward the teacher's family (search targets carry the teacher's logit shape), so architectures can't be compared on it. that is why the outcome target was used.
- under the full RL recipe (DAgger, then MARL PPO) a 78.5M-parameter CNN replica trailed graph-s by 7–12 points against F at a quarter of the training speed, and graph-m tied graph-s head to head (50.6%) at 2.3× the cost per step. step cost at batch 512: MLP 3 ms, graph-s 26 ms, CNN 120 ms; the graph net is launch-bound (about 200 small kernels per forward), which CUDA graphs fix at inference.

what the formal ablation should look like in v2. the harness already sweeps `model.preset` (`configs/experiments/arch-lineage.yaml` sweeps `graph-s`, `graph-m` and `mlp` over two seeds, but at 400M steps a cell is 6.5 h, so 12 cells is a week of GPU). the v2 row is the engine's padded seat-relative graph row, so the MLP is just a flat network over it (`configs/models/mlp.yaml`, padded dims only) and the CNN has to lay the BASE board out on a grid itself from the tile→node topology the row carries. an LSTM is the odd one out: it changes what the policy sees (a history instead of the latest state), so it is also the first step of the "how markov" idea in Section 3.

- ☑ port v1′s xdim CNN backbone to the v2 row (`src/catanrl/models/cnn.py`, preset `cnn`), BASE-map only like the MLP.
  - done 2026-10-17: v1′s `xdim-default` size (16.8M parameters per network, 15.1M of them in the projection after the convolutions), the tile/node/edge blocks gathered onto v1′s 11 × 21 plane that is laid out once from the engine's BASE map. deviations from v1, all documented in the module: GroupNorm instead of BatchNorm (so rollouts and evals run the same network; `norm: batch` restores v1), every block feature is a channel (47 at 2p) instead of v1′s 20 hand-picked planes, v2′s structured heads. a map with BASE's dims but another tile order (the `hexagon` generator) is refused at launch rather than silently scrambled, which a review caught.
- ☑ an LSTM policy over each seat's row history through pufferlib's `use_rnn` path (preset `lstm`): the trunk embedding feeds an LSTM cell whose state persists across env steps and resets at game ends.
  - done 2026-10-17: the `graph-s` trunk, an LSTM cell over its pooled embedding stepped at each of the seat's own decisions (waiting rows keep their state, a game's first decision starts from zero), the memory injected into both heads; 0.9M parameters. PPO replays the recurrence over segments × horizon from the stored rollout states (CleanRL's PPO-LSTM shape); a 5-epoch run at learning rate 0 reproduces the rollout's logits

exactly (KL 0, clip fraction 0). the native `POLICY` agent keeps one memory stream per seat of a game, so eval records don't depend on `num_envs`; MCTS, BELIEF and the server bridge refuse it for now (a sampled world has no decision history of its own).

- ☑ `configs/experiments/arch-ablation.yaml`: the current best recipe (Section 2) at the fair `private` level, one seed, 100M agent steps per architecture (about 1.6 h each for the graph nets and the mlp, the cnn slower; puffer-graph2 was at 86% vs F by then, and v1's gaps were 7–12 points, so a 100M-step cell separates them), selected on `select-2p`, reported on `heldout-2p` vs F and head to head against the graph net on the same boards. written and validated 2026-10-17; every cell is `shared: true` except the lstm, which has one trunk by construction.
- ☑ run it. ran 2026-10-18, 01:37 to 13:08, one invocation (`catanrl run arch-ablation`; the runner orders the cells because the head-to-head rows depend on `graph-s`).

1.1 what the formal ablation says

same recipe, same 100M agent steps, same boards, one seed, `private` level. "vs F" is the argmax policy on `heldout-2p` (5,000 games); "vs graph-s" is the same policy against the `graph-s` cell's best checkpoint on the same boards.

Table 1: the formal ablation (`configs/experiments/arch-ablation.yaml`, records `039b60f0`, `1cd2de87/073da498`, `76c88ebd/189e5e8d`, `8e2e23d8/5abf6f0c`). the aux row is `configs/experiments/aux-hidden.yaml` (`0e28d5cf/c6b7f682`), the same `graph-s` cell plus the hidden-feature head of Section 3. wall time includes the select evals; the mlp's 0.3 h is why it is the cheap baseline and the cnn's 7.5 h is why v1 abandoned it. live logs: the `arch-ablation` group on wandb.

arch	params	steps/s	wall	select-2p (best)	vs F	vs graph-s
`graph-s` (shared trunk)	0.54M	17.6k	1.6 h	92.5 (e750)	**90.7** 89.9–91.5	—
`lstm` (graph-s trunk + LSTM)	0.86M	14.8k	1.9 h	92.0 (e700)	**90.3** 89.4–91.1	**51.0** 49.7–52.4
`graph-s` + aux head (Section 3)	0.62M	17.1k	1.7 h	90.0 (e700)	89.1 88.2–89.9	47.7 46.3–49.1
`cnn` (v1's xdim)	16.8M	5.6k	7.5 h	81.5 (e700)	80.4 79.3–81.5	33.7 32.4–35.0
`mlp`	2.2M	108k	0.3 h	68.0 (e700)	64.9 63.6–66.2	18.1 17.0–19.2

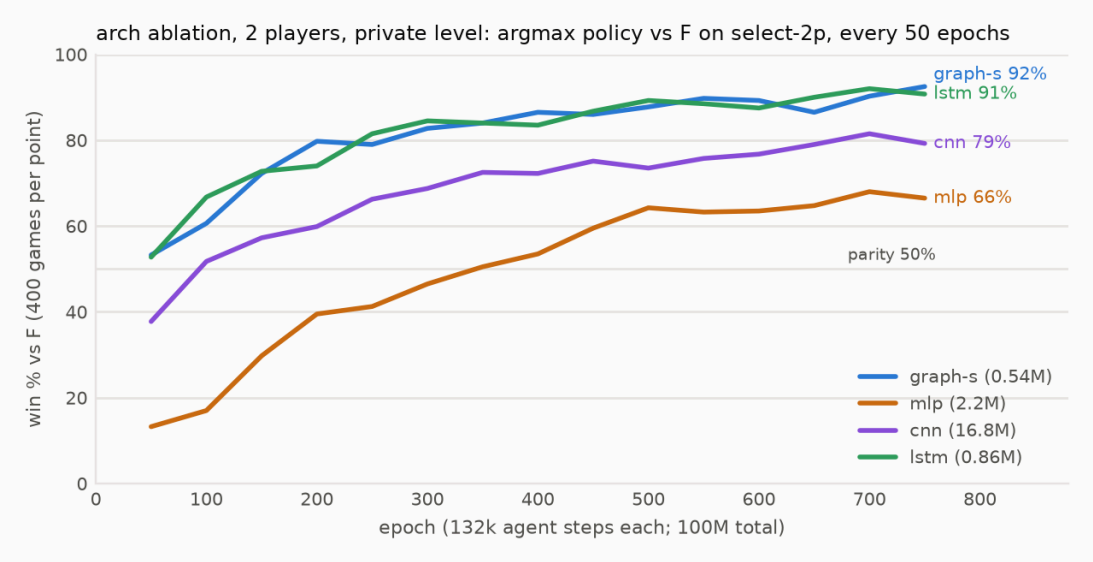


Figure 1: the four cells' selection curves on the same axis (argmax policy vs F on `select-2p`, every 50 epochs). the graph net and the lstm are indistinguishable from epoch 300 on; the cnn tracks ten points lower; the mlp never catches up.

so, with the informal ablation now formal: graph ≈ lstm > cnn > mlp, and the structured head is what matters. the lstm, which is the graph net with memory over the seat's own decisions, ties it head to head (51.0, a coin flip) at 1.6× the parameters and 0.85× the throughput, so at this budget a recurrent state adds nothing the current row doesn't already carry (more on this under the "how markov" idea in Section 3). the cnn, 31× the parameters and a third of the speed, is ten points behind against F and loses two of three games to the graph net; the mlp is 26 points behind and loses four of five. one seed, 100M steps, 2 players: enough to rank the families, not to separate graph from lstm, which would need the full 500M-step budget and more seeds.

2 current best recipe

implement engine in c, turn into pufferlib env, run for 1B steps, and get a really good shared-trunk critic + policy that beats `ValueFunctionPlayer` 90% of the time.

put the critic + policy inside a mcts harness, with a simple probabilistic belief-sampling determinization method to get another 5% h2h edge over the raw policy.

2.1 the recipe, concretely

- **engine.** cppanatron, a C++20 port of catanatron kept in lockstep parity with it (2–6 players, explicit maps of any shape). the observation encoder, the action codec, the threaded multi-game env, the hand tracker and the batched MCTS all live in C++, so training, search and deployment can't drift apart. one float32 row per (game, seat), seat-relative, padded to the env's maxima: `[global | players | tiles | nodes | edges | topology | action mask]`.
- **network.** `graph-s`: canopy nexus-v3 style heterogeneous message passing (node↔node, node↔tile, node↔edge), hidden 96, 3 layers, one shared trunk for the policy head and a per-seat value head (one sigmoid per seat = that seat's win probability; the acting row's prediction fills the waiting seats' values, so GAE on each seat's own trajectory costs one network call per game step). 541,540 parameters.
- **training.** pufferlib 3 PPO with two small patches (losses over acting rows only, a value-loss hook), pure self-play (every seat learns), muon at 5e-3 with a cosine to 5e-4, momentum 0.95, $\gamma = 0.999$ per decision, GAE 0.95, 2 update epochs, clip 0.2, entropy 0.005, prioritized 64-step segments, fp32. the batch shape is v1′s graph2 run exactly: 1032 games × 2 rows × 64 steps = 132,096 agent steps per epoch, 16 optimizer steps per epoch, 3786 epochs. `configs/experiments/puffer-graph2.yaml` lists every deviation from v1 and why.
- **search.** `MCTS:ckpt=…:sims=N:dets=K`: a batched N-player PUCT search (per-seat values backed up max^n, exact chance nodes for dice, dev-card draws and steals) over K worlds sampled from the hand tracker's belief at the root (`dets=K`; `oracle=true` searches the true game and is only a reference). `BELIEF:…:worlds=16` is the raw policy averaged over 16 sampled worlds, the fair way to play a network that was trained on more than its seat sees.

2.2 how things are measured

every number is on a held-out suite of boards that training never saw (`validate` checks the training pools against them): `heldout-2p` is 2,500 boards (seed 67) played in both seat orders = 5,000 games; `heldout-4p` is 1,000 boards (seed 68) × 4 rotations = 4,000 games. checkpoints are chosen on a disjoint `select-*` suite every 50 epochs, so the held-out number is never the one that picked the checkpoint. intervals are wilson 95%. in N-player games the number is slot 0′s *score share* (a win counts 1, a truncated game 1/N to every seat), so parity is 1/N. agents that use information a player doesn't have (a `full` policy, an oracle search) are marked; everything else is fair.

2.3 results so far

Table 2: the current best agents against F. v2 trains from scratch (no DAgger) and lands 5 points above v1's warm-started reference in 2p. the 2p `full` policy sees F's development cards; its belief-averaged version doesn't and loses nothing (95.06 vs 95.16), which is the same "hidden information is worth under a point" finding as v1. in 4p the shared-trunk network is trained and played at the `private` level, which is the only fair level past two players.

agent	sees	games	vs F	record
2 players, BASE, 15 vp (`puffer-graph2/full@e3350`, 500M agent steps, 7.95 h)				
raw policy, `full` obs	hidden cards	5,000	**95.16** 94.53–95.72	`ff8528a4`
raw policy, belief-averaged (`BELIEF`, 16 worlds)	fair	5,000	**95.06** 94.42–95.63	`67869e7a`
raw policy at the `public` level (`puffer-graph2/public@e3600`, same recipe, 7.95 h; no belief averaging needed)	fair	5,000	**94.96** 94.3–95.5	`dc8a611a`
MCTS 64 sims on the true game	hidden cards	500	94.8 92.5–96.4	`3fdae985`
MCTS 256 sims on the true game	hidden cards	500	95.2 93.0–96.8	`609e0aa7`
MCTS 1024 sims, 1 det., vs `BELIEF` 16 worlds (head to head)	fair	1,000	53.4 50.3–56.5	`04fd2c25`
MCTS 4096 sims, 1 det., vs `BELIEF` 16 worlds (head to head)	fair	500	55.2 50.8–59.5	`2aaf2eb7`
MCTS 1024 sims, 1 det., vs the raw `public` policy (`puffer-graph2/public@e3600`, head to head)	fair	1,000	54.9 51.8–58.0	`72819d1a`
MCTS 4096 sims, 1 det., vs the raw `public` policy (head to head)	fair	500	55.2 50.8–59.5	`0ef1b81e`
raw policy on one predicted row (`PREDICT`, `aux-hidden` predictor, Section 3)	fair	5,000	94.72 94.1–95.3	`b74d240b`
`PREDICT` vs `BELIEF` 16 worlds (head to head)	fair	5,000	50.4 49.1–51.8	`e7e34d28`
4 players, BASE, 10 vp (`ppo-4p/main@e3650`, 1B agent steps = 249M decisions, 9.3 h)				
raw policy vs F, F, F (score share, parity 25%)	fair	4,000	**78.55** 77.25–79.79	`e5b302ee`
two copies vs F, F (team share 93.2 = 40.45 + 52.75; the split is the slot order, Section 3)	fair	4,000	93.2	`b07e5bd9`
raw policy vs R, R, R	fair	4,000	100.0 99.9–100.0	`95570c74`
MCTS 1024 sims, 1 det., vs three raw copies (critic-fix net `lambda-critic-g1@e100`; share, parity 25%)	fair	400	27.0 22.9–31.6	`90a1f06c`
v1 reference (graph-s, DAgger + MARL PPO, 73.7M steps), for history				
raw policy, belief-averaged	fair	2,000	89.65 88.24–90.91	v1 report
MCTS 256, determinized	fair	1,000	90.5 88.5–92.2	v1 report
MCTS 1024 on the true game	hidden cards	1,000	92.7 90.9–94.2	v1 report

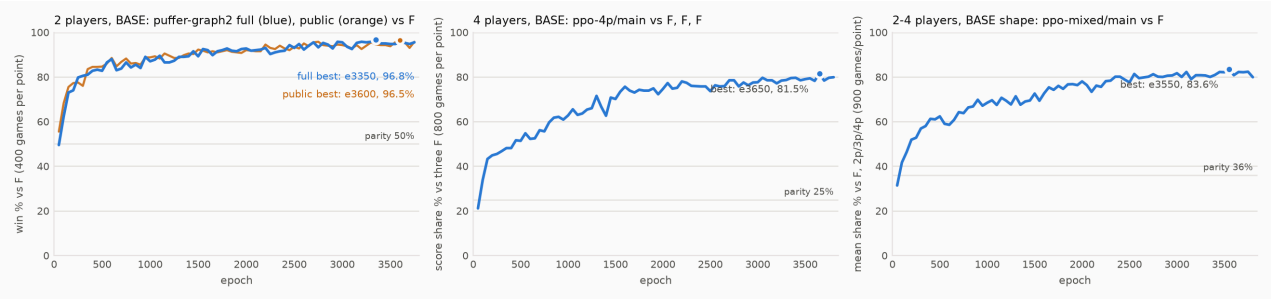


Figure 2: training-time selection curves (argmax policy vs F on the `select-*` suites, every 50 epochs). all runs keep improving through the second half of the cosine, so matched comparisons need the whole budget; the 2p `public` variant (orange) tracks `full` (blue) throughout. full per-epoch logs: puffer-graph2/full, puffer-graph2/public, ppo-4p/main and ppo-mixed/main on wandb.

the wandb runs were backfilled from `runs/*/metrics.jsonl` and the eval records after the fact (the trainer writes jsonl; wandb mirrors and never decides), so they hold everything the trainer logged: losses, entropy, KL, clip fraction, game length, seat win rates, the critic's calibration (`critic/value_sum_*`: the per-seat win probabilities should sum to about the share of games somebody wins), throughput and the select curve. `ppo-4p`'s critic summed to about 1.2 over its four seats against a realized 0.88, a bias from V-trace targets on waiting rows; `configs/experiments/ppo-4p-critic-fix.yaml` fixed it on 2026-10-18 (below). since 2026-10-17 there is a live mirror (`catanrl.wandb_mirror`, on whenever an experiment's `meta.wandb.project` is set; the record on disk stays the source of truth and every wandb failure is a warning) and `tools/wandb_backfill.py` re-syncs any run directory into the same wandb run, so a run launched without the mirror, like `ppo-mixed`, lands there too.

2.4 what is still missing for this section

- ☑ **the 5% search edge in v2, head to head.** v1 measured MCTS against the same network's belief-averaged raw policy on 1,000 games: 51.1% at 256 sims, 55.1% at 1024, 58.1% at 4096 (against F both are near the ceiling, which is why the vs-F rows above don't move). v2, same network, fair search (`dets=1`, worlds from the hand tracker) against `BELIEF` 16 worlds on `heldout-2p`, finished 2026-10-18: **53.4** 50.3–56.5 at 1024 sims (n=1,000, `04fd2c25`) and **55.2** 50.8–59.5 at 4096 (n=500, `2aaf2eb7`). so the edge is there and grows with sims, a point or two smaller than v1′s over a stronger raw policy; both seats of the pairing agree (first seat 55.0 / second 51.8 at 1024). cost: 11.5 h of gpu for the 1,500 games at 128 games in flight, which is what parks alphazero at these sim counts (Section 3). the `public` network's own rows (next item) repeat it over its raw policy: 54.9 at 1024, 55.2 at 4096, in 5.0 h.
- ☑ **the fair 2p variant.** `puffer-graph2/public` (the `public` level: in 2p every resource movement is seen by both seats, so the opponent's exact hand is a perfect card counter's knowledge, and a `public` policy is fair without belief averaging; what `full` adds over it is only the opponent's dev cards in hand) ran 2026-10-18 18:03 to 2026-10-19 02:06: the same recipe and budget as `full`, the same 7.95 h at 17.8k steps/s, best checkpoint e3600 (select 96.5 against `full`'s e3350 at 96.8; the two select curves lie on top of each other, Figure 2, mean over epochs ≥ 3000 94.4 vs 95.1). final: **94.96** 94.3–95.5 vs F on `heldout-2p` as a plain fair `POLICY` (`dc8a611a`, 5,000 games), against the `full` variant's belief-averaged 95.06 94.4–95.6 and its oracle 95.16, so the one thing `public` cannot see, the opponent's dev cards, is worth nothing measurable in 2p (the aux head found the same from the other side: the hand is inferable, the dev cards are not, and the policy did not need them). one forward pass, no sampled worlds, no tracker: this is the bot to serve raw. its own search rows ran 02:07 to 07:10 (5.0 h for the 1,500 games, half of what the `full` finals took: the opponent here is one forward pass, not sixteen, and 256 games were in flight instead of 128): MCTS 1024 sims, one world from the tracker, beats the raw `public` policy **54.9** 51.8–58.0 on 1,000 games (`72819d1a`, 1.5 h) and 4096 sims **55.2** 50.8–59.5 on 500 (`0ef1b81e`, 3.6 h), the same edge the `full` network has over its belief-averaged self (53.4 / 55.2 above). both seats agree (the searcher 60.6 in seat 0 and 49.2 in seat 1 at 1024, the raw policy 50.8 / 39.4), with a first-seat advantage under both: seat 0 wins about 56% of these games whoever sits there.
- ☑ **the critic fix.** `ppo-4p`'s critic summed its four seats' win probabilities above the realized sum (about 1.2 against 0.87 at epoch 300; 0.93 against 0.90 by the end), from two causes found on cpu replications: V-trace ratios on the waiting rows of each sample (only the acting seat's row gets min(ρ, 1), the other three get 1, so each seat's target values the game as if that seat alone played the improved policy, and the four one-sided gains compound through bootstrapping) and muon's fixed-size update of the 1 × 128 value output layer (the rollout network lands on the high side every epoch). `configs/experiments/ppo-4p-critic-fix.yaml`, run 2026-10-18 (four arms × 15 min): 100-epoch fine-tunes of `ppo-4p/main@latest` at its final learning rate with lambda-return critic targets (every ratio 1; the policy's advantages stay V-trace's), at three discounts and one with the output layer on AdamW. every arm puts the seat sum on the realized sum within about 20 epochs and keeps it there (Figure 3): `lambda` 0.906 against 0.899 realized at γ = 0.999 (from 0.920 against 0.914 in the first epochs), `lambda-critic-g1` 0.998 against 1.000 with the critic undiscounted, `lambda-g1` 1.000 against 1.000 with policy and critic undiscounted, `lambda-adamw` 0.906 against 0.898. the rollout-vs-trained gap falls from 0.004 to 0.000 in all four, so the targets alone remove muon's phase lock and the AdamW arm buys nothing. per-slot calibration at the end of `lambda-critic-g1`: the seat to move predicted 0.284 against a realized 0.276, the other three within 0.004. the policy does not move in 100 epochs: select-4p 79.9 at the start and 80.4 / 79.9 / 80.4 / 79.1 at the end of the four arms (n=800 per point, ±2.8), clip fraction, entropy and game length (79–80 turns) unchanged, and the undiscounted policy plays games of the same length, so γ = 1 changes nothing the policy optimises at this horizon. recommendation, applied to the defaults on 2026-10-19: `critic_targets: lambda` and `critic_gamma: 1` in `configs/train/ppo.yaml` for every run from now on (per-seat win probabilities that sum to one are what max^n backups want); the experiment yamls with stored runs pin the previous values, so their config hashes and their records are unchanged. the first from-scratch run with the new critic is `aux-hidden-4p`'s plain arm (Section 3). live logs: the `ppo-4p-critic-fix` group on wandb.
- ☑ **the 4p search rows.** `configs/experiments/search-4p.yaml`, run 2026-10-18 (71 min for 400 games): `MCTS 1024 dets=1` with the critic-fix network (`ppo-4p-critic-fix/lambda-critic-g1@e100`, undiscounted per-seat values that sum to one, the same raw policy as `ppo-4p` to within noise) against three copies of that network's raw policy on `heldout-4p`, 100 boards × 4 rotations: a 27.0 22.9–31.6 share (`90a1f06c`) against the raw copies' 24.3 and a

parity of 25. so the search edge in 4p is about two points of share (8% relative), the same order as the 2p edge's 3.4 points of win rate, and 400 games cannot separate it from zero; 2,000 games would (about 6 h): queued as `configs/experiments/search-4p-2000.yaml` (the same table on 500 boards × 4 rotations), behind the public run's finals. the searcher took 32.0 in seat 0 against 24–27 in the other seats and the raw copies 25–28 in seat 0, so a small first-seat advantage sits in these numbers too (no F at the table, so no F targeting; the four rotations are summed).

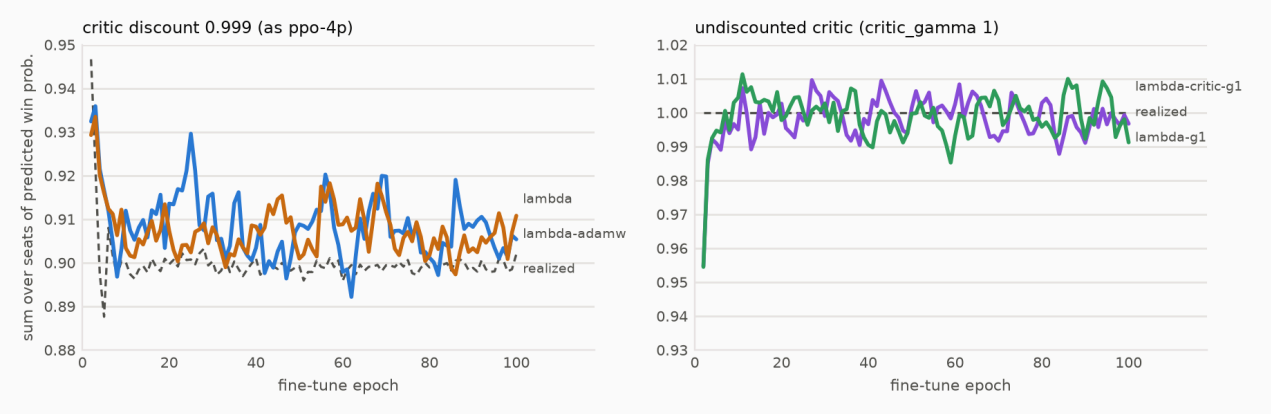


Figure 3: `ppo-4p-critic-fix`: the four seats' predicted win probabilities summed, per fine-tune epoch, against the realized sum (dashed: about 0.90 at $\gamma = 0.999$, 1 undiscounted). lambda-return targets put the sum on the truth in every arm within about 20 epochs; the AdamW output layer changes nothing.

3 exploring ideas

the list, with a box per idea; sub-bullets are the thinking so far and what has actually been done. running experiments are dated.

- ☐ **training alphazero** (currently agreement is really high, doesn't seem to yield many gains)
  - the loop exists (`train.algo: alphazero`, `configs/experiments/az-from-ppo.yaml`): self-play through the batched search with gumbel roots, a replay buffer of (row, visit-count policy, per-seat outcome), warm-started from a PPO checkpoint, judged head to head against the network it started from (against F a strong policy is at its ceiling, so vs-F can't see the gain).
  - the one run so far, from a PPO network with 64/16-simulation targets, degraded the policy; no bug found, and shallow search targets can't improve a policy that is already better than a 64-sim search of itself. the next attempt needs ≥ 1024-sim targets, which at 5 s per game-move-batch is expensive: parked until the search edge above is measured, because that number says how much there is to distill at all.
- ☑ **training one model to train for up to `n` players and arbitrary maps** (should be possible by using masked values for numeric features and GNN already handles arbitrary hex boards)
  - the harness already does the masking: the row is padded to the env's maxima with a validity bit per slot, absent seats are all-zero, the global block one-hots the number of seats, and the per-seat value head predicts every present seat. `env.scenarios` draws each game from a weighted list of (player count, rules, map generator).
  - **running since 2026-10-17 02:14** (live on wandb: ppo-mixed/main, re-synced every 20 min): `configs/experiments/ppo-mixed.yaml`, `ppo-4p`'s recipe over six equally weighted scenarios: 2, 3 and 4 players on BASE and on `hexagon` boards (BASE shape, shuffled numbers, generated harbors), same decisions per epoch as `ppo-4p` (3800 epochs, about 10 h). blob-shaped boards of 15–30 tiles (`random-4p`) stay out of training because they would double the row (6316 floats vs 3202) and the memory; the run is evaluated zero-shot on them (`heldout-random-4p`). finals: vs F on `heldout-2p`, `heldout-4p`, `heldout-mixed`, `heldout-random-4p`, and head to head with the specialists (`ppo-4p` on 4p boards, `puffer-graph2`'s belief policy on 2p boards). the question is whether one network matches the specialists at equal decisions; if the blob number is poor, the follow-up trains on blobs with fewer games in flight; 5–6 players (the extension map) comes after, since the engine lacks the special building phase.
  - **result** (finished 2026-10-17 13:54 after 11.5 h; best `e3550` by `select-mixed`, 83.6; 1B agent steps = 249M decisions, the same as `ppo-4p`). score share vs F: 2p 92.3 91.6–93.0 (`b2690265`, n=5,000; the 2p specialist has

95.2), 3p 81.1 79.0–83.0 (n=1,500, parity 33), 4p 77.0 75.7–78.3 (`0f448f68`, n=4,000; the 4p specialist has 78.6). zero-shot on blob maps it never saw (`heldout-random-4p`, 15–30 tiles): 73.9 72.5–75.3 (`7c17d0ae`, n=4,000). the mixed suite as a whole: 79.8 (`9b470a17`, n=4,500).
    - head to head it loses to both specialists: 45.7 44.4–47.1 vs `puffer-graph2`'s belief-averaged policy in 2p (`340ce1db`, n=5,000), and a 39.3 37.8–40.9 share vs `ppo-4p`'s 53.0 with two F at the table in 4p (`32aab9f3`, n=4,000). **correction (2026-10-18):** the 4p number is not evidence. two identical copies of `ppo-4p` at the same table split 40.45 / 52.75 (`b07e5bd9`), because slot 0 always moves right after the F in slot 3 and F robs the seat after it (the tracker item below has the diagnosis). the swapped order (`configs/experiments/slot-order.yaml`, `0903fae0`, n=4,000, run 2026-10-18: the specialist 42.65 in slot 0, the generalist 50.85 in slot 1) settles it. averaged over both orders the generalist's share is 45.1 against the specialist's 47.8, a paired −2.7 pp −4.7–−0.8 per game, so the 4p specialist does win, by three points rather than fourteen.
    - reading: one network handles any seat count and board shape within 2–3 points of a specialist against F, and the graph net transfers to board shapes it never trained on nearly for free (73.9 on blobs vs 77.0 on BASE), but every setting got a third of the decisions, the specialists win the direct matches, by four points in 2p and three in 4p. so this is a capacity-vs-budget question: (my suggestion, not queued) either give the generalist the specialists' per-setting budget (3× the decisions, about 35 h) or fine-tune it per setting for a few hundred epochs; if the gap closes, the generalist is the model to ship.
- ☐ **train the generalist on blob maps too**, with fewer games in flight to fit the 6316-float rows, now that zero-shot says the architecture copes; the test is `heldout-random-4p` against the 73.9 above.
- ☐ **5–6 players** (the extension map): needs the special building phase in the engine first; after that it is one more scenario in `ppo-mixed`.
- ☐ **adding trading to the engine**
    - the rules side is done: cppanatron implements domestic trades (offer, accept, reject, confirm, cancel) in parity with catanatron. what is missing is everything in front of it: the map-agnostic action space (`GraphActionSpace`) leaves domestic trades out, the env never offers them, and the encoder has no trade-negotiation prompt. so "adding trading" means an action-space extension plus the env, the mask and the trade prompts in the row.
- ☐ **allowing the model to "learn trading"**
    - do not increase action space explosively, either
    - have the model predict the player trade action and a float for each of the 5 types of elements, for both what they give and what they get (11 extra options in the action space), and round to the nearest integer for each of the 10 floats, or
    - have the model predict the player trade action and a float for each of the 18 possible integer choices for each of the 5 types of elements, for both what they give and what they get (181 extra options)
    - i suspect that training while allowing this from scratch will likely not result in the model learning to trade well. instead, we should first train a competent model using the ppo recipe, then take random game states, try applying all every possible legal trades to the current state, and evaluate which trades modify the game state such that the new game state is better for both sides of the trade, according to the critic model
    - plan, in that order: (1) the critic-side analysis first, because it needs no action-space change: sample positions from the trained 2p and 4p networks' self-play, enumerate every 1-for-1 and 2-for-1 trade between the seat to move and each opponent, apply it to a clone of the position and read both seats' win probabilities from the critic; report how often a trade raises both, by how much, and what the typical trade is. that says whether trading is worth an action space at all under this critic. (2) if it is, the compact head (option one above) for the proposer and a yes/no for the responder, trained from the ppo checkpoint with the trade actions masked in at a low entropy bonus.
    - (1) done 2026-10-17: `tools/trade_oracle.py`, results in `report/data/trade-oracle-{ppo-4p-main,puffer-graph2-full}.json`. 3,000 positions each from the network's own self-play (temperature 1, about 3.4 positions per game, 884 / 916 games), every 1:1, 2:1 and 1:2 both hands can pay (about 73 per 4p position), applied through the engine's own offer/accept/confirm path, both seats' win probabilities read from the acting row (the only row shape the critic was trained on; a first version also read the partner's own row and a review showed that to be off-distribution noise). the critic's own noise floor is reported next to every number: its per-seat values don't conserve under a trade (median residual 1.1 pp in 4p, 0.5 pp in 2p), so the 0.5 pp and 1 pp rows are within noise and the 2 pp row is the one to read.

  - 4p (`ppo-4p/main@best`, private): a trade that raises *both* seats' win probability by ≥ 2 pp exists in 6.2% of positions [5.4, 7.2]; by ≥ 1 pp in 25.8% [24.2, 27.5]; by ≥ 5 pp in 0.4%. the best mutual trade gains the mover 1.2 pp and the partner 0.9 pp on average, and costs the two seats not in it 1.1 pp. 92% of mutual trades are 2:1 (the mover gives two; at the private level the mover's row sees the partner's hand only as a size, so this is partly what the critic can see). the mover's selfish best trade gains it 7.8 pp and costs the partner 3.7 pp on average: trades are mostly zero-sum, as expected. most common mutual trade: two wood for an ore.
  - 2p (`puffer-graph2/full@best`, full information): ≥ 2 pp in 0.3% of positions [0.1, 0.5], ≥ 1 pp in 4.3%. two-player catan is nearly zero-sum for this critic, so trading would be an action space that is almost never used.
  - so: worth an action space in 4p (a modest, real pool of 2:1 trades the critic likes for both sides, each costing the other two seats), not in 2p. step (2) waits for the mixed run and your call on the head (option one above is the default).
- ☐ **build the best possible full-information tracker for any number of players** that resolves hidden information (e.g. a robber steal between 2 other players) if either of the other players reveals what happened in the trade via a new action
  - the only hidden information are dev cards and robberies.
  - for example: let us be player A. at the start, we know player B's hand is a wood and a sheep. player C's hand is a brick, an ore, and a wheat. if player B robs player C, and then buys a road, we can resolve that player B now has a sheep, while player C has an ore and a wheat. or if player C robbed player B, and then bought a dev card, we know that player C has a brick, and player B has a wood.
  - another example: let us be player A. we have 6 knights played, player B has 6 knights played, and player C has 1 knight played. then, if player C plays another knight, we know that there are no more knights in anyone else's hand, or in the development card deck, because player C has just played the 14th (and last) knight. we should then never account for any knights in the game. this is important because say if player B has largest army, we know we should stop trying to contend for largest army because we can't get buy any more knight cards. obviously this goes way further than this, since we should be updating our prior on our belief set for the sampling determinization method every time someone plays a dev card.
  - where the current `HandTracker` (C++, `engine/cppanatron/include/cppanatron/tracker.hpp`) stands against these: it keeps, per seat, a known lower bound per resource plus a count of unknown cards, and one global pool of unknown cards derived from the bank, so both robbery examples above should already resolve (when B pays wood + brick the brick comes out of B's unknown card, the pool is left with ore + wheat, and C's two unknown cards can only be those). the knight example is covered on the sampling side (`unseen_development_cards` = deck − my hand − every played card, so after the 14th knight no sampled world holds one) and the row carries every seat's played cards per type plus my own hand, so the network can count too. what it does *not* do: it keeps no prior over *which* card a hidden steal took (a steal from a hand of brick, brick, ore is sampled as brick or ore with equal weight instead of 2:1), it drops evidence from what seats chose not to do, and it is uniform over consistent worlds only up to those two approximations.
  - plan: pin the two examples above as tests, then an exact-posterior tracker that keeps a weighted set of hypotheses over the hidden steals (pruned by every later payment, merged when they reach the same hands, capped with a fall-back to today's pool) and samples worlds from it; measure (a) agreement with brute-force enumeration on simulated 3p/4p games and (b) `BELIEF` and `MCTS dets=K` vs the current tracker on `heldout-4p`.
  - done 2026-10-17, the tracker and (a): both robbery examples and the 14th-knight example are pinned as engine tests, and the existing pool tracker already resolves all three. the exact mode (`tracker=exact` on `BELIEF` and `MCTS`; `pool` stays the default) keeps weighted hypotheses over hidden steals, branched by the victim's count of each kind, pruned by every later payment, merged when they reach the same hands, capped at 4096 (the most seen in 424 simulated 2–6p games: 748; 2 on average), falling back to the pool past the cap. against brute-force enumeration on 160 positions the exact belief is identical (total variation 0.0); the pool tracker's sampled deal is off by 0.12 (random play) to 0.18 (value play) total variation on average and up to 0.9 in the worst position, so the approximation is real, not just theoretical. sampling a world costs the same (about 12 µs). (b), the `heldout-4p` evals, is queued on the gpu.
  - (b) is `configs/experiments/tracker-modes.yaml` (2026-10-18): the two tracker modes of `ppo-4p`'s belief policy, 16 worlds each, at one 4p table with two F, 500 boards × 4 rotations, so every game is a paired comparison.

runs after `aux-hidden`. if exact beats pool there, the same pairing with `MCTS:sims=256:dets=4` follows (about ten times the cost).

- **2026-10-18, (b) result: void at this level, and it exposed an eval artifact.** the table ran in 3 minutes (2,000 native games, record `79147906`) and gave exact 40.4 38.2–42.5 against pool 52.7 50.5–54.8, a paired −12.3 pp [−16.5, −8.1], the same at every seat. that is not the tracker. a cpu replay of 1,208 decisions (diagnostic scripts in the session scratchpad, not in the repo) showed that at the `private` level every sampled world encodes to the true row (0 of 4,832 worlds per mode differ in any feature: the encoder zeroes exactly what a world could change), so `BELIEF` with either tracker is the argmax policy, and the record is game for game identical to `ppo-4p`'s own `[POLICY, POLICY, F, F]` control (`b07e5bd9`: 40.45 / 52.75). the gap is the lineup. the suite seats `lineup[(seat + r) mod 4]`, so slot 0 always moves right after the F in slot 3, and F (`enemies=next`) robs and blocks the seat after it: in the replayed games the F in slot 3 put the robber on slot 0's tiles in 280 of 280 moves, slot 0 was stolen from by an F 263 times and slot 1 39 times. with `F:enemies=all` the difference flips to about +7 pp, so even then the two learner slots are not exchangeable: a paired 4p table needs both slot orders (or the sum, as the "two copies" row in Table 2 does). this voided the 4p head to head of `ppo-mixed` against `ppo-4p` as first reported; `slot-order` ran the swapped order and the `ppo-mixed` item above has the corrected number. what survived: on those 1,208 4p decisions the exact deal reproduces the other seats' true resource hands in 72.0% of worlds, the pool deal in 60.5%, with no fall-back (1–71 hypotheses), and the exact-mode env is bit-identical to the pool-mode env. the tracker can only matter to an agent that reads the sampled hands: in 4p that is `MCTS` with dets (search plays out steals and builds with them) or a full-level 4p net played through `BELIEF` (none trained; `ppo-4p` is private, and whether 4p should train at `full` and play through `BELIEF` as 2p does is an open question), and in 2p never (no hidden steals, the two trackers coincide). box: `MCTS:sims=256:dets=4` exact vs pool with `ppo-4p`'s net, both slot orders, 200 games (about 3 h; after the critic fix is read, since search backs up the critic). queued 2026-10-19 as `configs/experiments/tracker-search.yaml` (2 × 50 boards × 4 rotations = 400 games).
- **2026-10-19, the search pairing: a tie, game for game.** `tracker-search` ran 07:11 to 07:38 (27 min for the 400 games, not 3 h: the searchers batch their leaves across the 256 games in flight): `MCTS:sims=256:dets=4` with `ppo-4p/main@e3650`'s net, exact against pool at one table with two F, 50 boards × 4 rotations in each slot order. exact 44.5 37.8–51.4 against pool 52.5 45.6–59.3 with exact in slot 0 (`89915b43`), pool 44.0 37.3–50.9 against exact 52.0 45.1–58.8 with the orders swapped (`139796c6`): 193 wins each out of 400, a paired difference of 0.0 pp [−3.2, 3.2] over the two orders (bootstrap over the 200 board × rotation pairs), and the slot after the F loses 8 points in both orders (44.3 against 52.3: the artifact above, now on the searcher). the pairing is tight because the two games of a pair are mostly the same game: on the same dice the same seat won in 186 of the 200 pairs, in the same number of turns in 169, with the same number of decisions per seat in 161, so at 4 worlds × 256 sims the sampled hands rarely differ enough to change the search's choice; the 14 pairs that did change hands split 6 to pool, 4 to exact, and 4 where exact won one game and an F the other. verdict for the item: the exact posterior is right (identical to enumeration, where the pool deal is off by 0.12–0.18 total variation) and costs nothing, but no agent in this repo plays differently for it: the private-level policies cannot see it, and search at the sim counts we can afford does not act on it. it stays as the default-off `tracker=exact` option; what would read it is a full-level 4p net played through `BELIEF` (the open question above) or search with many more worlds, neither queued.

- ☑️ **train the model to predict the full-information observation from the public-information observation** instead of sampling from belief
  - the natural target already exists: the `full` row of the same position (the env can encode any level), so this is an auxiliary head on the shared trunk predicting the hidden features (opponents' resource and dev-card counts) with a masked regression loss, trained alongside PPO at the `private` level. the test is whether the policy's score moves, not the auxiliary loss. after the mixed run.
  - implemented 2026-10-17 (`model.args.aux`, `train.aux_coef`, experiment `configs/experiments/aux-hidden.yaml`). the hidden set is exactly the 15 per-seat features the encoder zeroes for other seats below the level that shows them (5 resource ratios, actual VPs, 5 dev-card counts, 4 playable flags), pinned against the encoder by a test; the targets come from the `full` players block the row already carries for a critic at another level (`critic_level: full`), which the shared trunk never reads (a test perturbs that block and checks logits, values and the predictions are bit-identical), so the policy stays fair and evals encode only the `private` row. loss: masked MSE in the encoder's feature scale over present opponent seats on acting

rows, coefficient 0.3 from the measured gradient ratio against ppo's loss. with the coefficient at 0 the training is bitwise the plain recipe. the run is the arch ablation's `graph-s` cell plus the head (100M steps, 2p, private), judged vs F and head to head against that cell on `heldout-2p`; in the gpu queue behind the ablation. per-feature R^2 is logged (`aux/*`) so the report can show what the head actually infers (hands vs dev cards).

- **2026-10-18, result:** the head does not move the policy. the aux cell scores 89.1 88.2–89.9 vs F against the plain cell's 90.7 89.9–91.5, and 47.7 46.3–49.1 head to head against it on 5,000 games (records `0e28d5cf`, `c6b7f682`; the row is in Table 1). same throughput (17.1k vs 17.6k steps/s, 1.7 h), same critic at the end (explained variance 0.930 vs 0.928), same entropy, so at coefficient 0.3 the loss acted as a regulariser that cost about a point and a half and nothing else. what the head learned is the useful part (R^2 per feature over the last 50 epochs, from the logged `aux/mse_*` and `aux/var_*`): the opponent's hand is nearly determined by the public row already, resources 0.90–0.99 (wheat 0.99, ore 0.96, sheep 0.96, brick 0.93, wood 0.90) and the actual vp count 0.99; in 2p the tracker's known cards, the hand size and the production history leave little to infer. dev cards in hand are the genuinely hidden part and stay hidden: victory point cards 0.85 (a card bought and never played is a vp card, the net works that out), knights 0.60, monopoly / road building / year of plenty 0.10–0.22, the playable flags the same. so in 2p the trunk had nothing new to encode, which is the honest reason the policy did not move. two things stay in the box: (i) the same cell at 4p, where steals and the robber hide more and the resource R^2 should drop (queued 2026-10-19 as `configs/experiments/aux-hidden-4p.yaml`: `ppo-4p`'s recipe at 750 epochs, matched to the 2p cell in decisions, as a two-arm sweep, plain and aux, with the head to head in both slot orders); (ii) the literal reading of the item, act on the prediction instead of sampling worlds: run the full-information policy on the private row with the 15 opponent features filled in by the aux head (`PREDICT`, eval only, minutes of gpu) and pair it against `BELIEF:worlds=16` of the same checkpoint. (ii) is cheap and is the next worktree.
- **2026-10-18, (ii) built and measured.** `PREDICT:ckpt=<full policy>:predictor=<aux checkpoint>` (eval only; `configs/experiments/predict-vs-belief.yaml`): the full-information network `puffer-graph2/full@best` acts on the true game's private row with the opponent's 15 hidden features overwritten by `aux-hidden/main@best`'s prediction, two forward passes where `BELIEF` runs sixteen. fair by construction, pinned by tests: the private row plus the fill is bitwise the full row, and with the true features in the head's place the agent reproduces the full-information policy's actions exactly. on `heldout-2p`, 5,000 games a row, 18 minutes for all four: 94.7 94.1–95.3 vs F (`b74d240b`; `BELIEF` 95.1, the oracle 95.2), 50.4 49.1–51.8 head to head against `BELIEF` 16 worlds (`e7e34d28`), and 49.9 48.5–51.3 with the predictions snapped to the encoder's grid (`a939fa5f`). so in 2p one predicted row is worth sixteen sampled ones, which fits the R^2 table above: what the predictor gets right (resources, actual vps) is what the tracker's worlds agree on anyway, and what it cannot know (dev cards) averages out either way. the fourth row, 63.1 61.8–64.4 against the private-level `arch-ablation/graph-s` (`751509ea`), is confounded by budget (100M steps against `puffer-graph2`'s 500M) and says nothing about the fill by itself. still boxed: the same measurement in 4p needs a full-level 4p net and a 4p predictor (neither trained), and a predictor head on the full-level network's own trunk would make it one network again.

- ☐ **define different scopes for how "markov" a model decision is**
  - right now, we only feed the model the latest view of the game state as its input
  - we could instead train a sequential model on the entire game state history, e.g. transformer or lstm
  - this would allow the model to learn things like "if someone didn't play a knight 2 moves ago when one of their high production tiles was blocked, then they probably don't have a knight among their dev cards"
  - this uses many orders of magnitude more compute, but would be only way to get "perfect" bot
  - first step is the LSTM arm of the arch ablation (a recurrent state over the seat's own row history is the cheap end of this scope); a transformer over the whole history is the expensive end and only makes sense if the LSTM beats the markov graph net at `private` level. the lstm class exists since 2026-10-17 (section 1); its 100M-step cell is in the ablation queue.
  - **2026-10-18, the lstm arm:** it ties the markov graph net head to head, 51.0 49.7–52.4 on 5,000 games, and against F (90.3 vs 90.7), at `private` level where there is hidden information to remember (Table 1). so at 100M steps in 2p, memory over the seat's own row history buys nothing measurable: the row already carries played cards, hand sizes and the robber history the net needs. the transformer end of this scope is parked unless the lstm shows a gap at the full budget or in 4p, where more is hidden; the cheap next check is the lstm at 4p against `ppo-4p`'s recipe.

- ☐ **turn this into an LLM RL env/eval** (e.g. harbor task) and RL a small language model on it, and eval frontier models on how well they play catan
  - `catanllm` (the CS 159 project) already benchmarks LLM players through openrouter against catanatron bots with a few prompting strategies; v2′s game server (protocol 5: committed seeds, signed moves, verifiable transcripts) can seat an LLM client against our bots fairly. an eval is cheap (api cost, no gpu); RL-ing a small model is not on this box.
  - the eval half is built (2026-10-17): an `LLM` agent spec (`LLM:model=<openrouter id>:strategy=plan|basic|strategy|search|rules:thinking=true`) that plays through the catanatron harness next to `POLICY` and F, with `catanllm`'s five prompting strategies ported over. the prompt is public information only (opponents as hand sizes, dev-card counts, played cards and visible points; the original renderer leaked every seat's dev cards, which a test now pins), a stub model plays the same games for free so the whole thing is tested without an api key, and real-model records are marked non-replayable so a report refresh can't quietly spend money. `configs/experiments/llm-eval.yaml` is a 16-cell sweep (four models × plan/basic × vs F / vs `puffer-graph2`′s policy, 20 boards both seats) with placeholder model ids. `catanrl llm-cost` prices it from stub games: a 2p game to 15 points prompts the model about 20 times at 2–8k tokens a prompt, so a 50-game cell is about $16–26 with gpt-5 or sonnet-4.5 prices as a lower bound, and a model that actually plays well lasts longer and is prompted 70–90 times, so budget 3–4× that. waiting on an api key and a budget; the RL half is not on this box.
- ☐ **turn this task of making a strong catan bot an LLM RL env/eval**, and there is a good chance the next frontier models will become amazing at creating better catan bots.

4 log

- **2026-10-14.** `puffer-graph2/full` finished (7.95 h): 95.16% vs F on `heldout-2p`, best checkpoint e3350 by `select-2p`. `ppo-4p/main` launched right after.
- **2026-10-15.** `ppo-4p/main` finished (9.3 h): 78.55% score share vs three F, 100% vs three R. both runs backfilled to wandb. the queue's third job (`puffer-graph2/public`) never started.
- **2026-10-17.** this write-up. launched `puffer-graph2/full`'s missing search finals (MCTS 1024 and 4096 sims, one determinization, vs the belief-averaged policy) and `ppo-mixed` (2–4 players, BASE-shaped boards); the two shared the gpu badly (training at half speed, search at zero finished games in ten minutes), so the finals were stopped and go back in the queue behind the training run: one gpu job at a time from now on. started worktrees for the CNN port, the LSTM policy, the exact tracker, the trade oracle and a wandb mirror; all five were implemented, reviewed and merged into one integration branch the same day (main takes it once the gpu is idle, because the eval provenance hashes the code on disk and a live run refuses to continue under changed code). `ppo-mixed` backfilled to wandb and re-synced every 20 minutes. later the same day: the aux head (predict the hidden features) implemented and verified in a sixth worktree; the integration branch now also mirrors every real run to wandb by default. evening: the LLM player (eval half) built and verified in a seventh worktree. `ppo-mixed` finished at 13:54 (11.5 h): 92.3 / 81.1 / 77.0 vs F in 2p / 3p / 4p, 73.9 zero-shot on blob maps, loses head to head to both specialists (Section 3). the search finals started right after.
- **2026-10-18.** the search finals finished at 01:31 (11.5 h): MCTS beats the belief-averaged policy 53.4% at 1024 sims and 55.2% at 4096. main merged with the integration branch (tracker, cnn, lstm, trade oracle, wandb mirror, aux head, llm player), engine rebuilt, `arch-ablation` launched at 01:37 (graph-s, then mlp, cnn, lstm). the repo went public on github the same morning (submodule branches pushed to `mcrco/cppanatron`, `mcrco/catanatron` and a `mcrco/PufferLib` fork). the ablation finished at 13:08: graph ≈ lstm > cnn > mlp (section 1). `aux-hidden` launched at 13:11 and finished at 14:53 (1.7 h): 89.1 vs F and 47.7 head to head against the plain cell, so the hidden-feature head is a small regulariser and nothing more in 2p (Section 3). `tracker-modes` ran 14:53–14:57: exact 40.4 / pool 52.7 is a slot-order artifact, not the tracker (diagnosed on cpu the same afternoon; the `ppo-mixed` 4p head to head has the same flaw, `slot-order` queued). the critic-fix sweep ran 14:58–16:00: lambda targets put the seat sum on the realized sum within 20 epochs in every arm, the undiscounted critic sums to 1.00, the policy unchanged (Section 2). `slot-order` ran at 16:00 (one minute, 4,000 games): over both orders the 4p specialist beats the generalist by 2.7 pp paired, not 14. `search-4p` ran 16:33–17:45 (400 games): MCTS 27.0 share against a parity of 25 in 4p. the `PREDICT` agent (act on the predicted hidden features) was built, reviewed and verified in an eighth worktree during the afternoon; main took it at 17:44 and `predict-vs-belief`

ran 17:45–18:03: one predicted row ties sixteen sampled worlds (50.4 head to head, 94.7 vs F). `puffer-graph2/public` launched at 18:03 (8 h of training, then its finals).

- **2026-10-19.** the critic fix became the default (`critic_targets: lambda`, `critic_gamma: 1`; the older yamls pin the old values so no stored run is re-keyed), and three experiments were queued behind the public run's finals, cheapest first: `tracker-search`, `search-4p-2000`, `aux-hidden-4p`. `puffer-graph2/public` finished training at 02:06 (7.95 h, as `full`): 95.0 vs F as a plain fair policy, the same as `full`'s belief-averaged 95.1, so the hidden dev cards buy nothing in 2p (Section 2); its MCTS finals started at 02:07 and finished at 07:10 (5.0 h, half the estimate: a one-forward opponent, 256 games in flight): MCTS 54.9 at 1024 / 55.2 at 4096 over the raw `public` policy, the `full` network's edge again. `tracker-search` ran 07:11–07:38 (27 min, 400 games): exact and pool tie exactly, 193 wins each over both slot orders, because the two games of a pair are mostly the same game (Section 3). `search-4p-2000` started at 07:38.